

TJSC 2026 - Analista de Sistemas

Arquitetura de Sistemas e Integração

Aula 2 - MVC e DDD

Padrões arquiteturais Model-View-Controller e Domain-Driven Design

Disciplina	Arquitetura de Sistemas e Integração
Aula	2 de 11
Recorte do edital	1.3 Padrões arquiteturais MVC e DDD
Banca	FGV - Fundação Getulio Vargas
Objetivo	Converter vivência prática em APIs, Java/Quarkus, Angular e integração corporativa em acerto de prova
Estilo	Teoria objetiva e aprofundada, quadros comparativos, pegadinhas e 20 questões autorais com gabarito separado e comentários completos

Material autoral, orientado ao edital vigente do TJSC 2026 e calibrado pelo padrão de cobrança da FGV. As questões são inéditas e não reproduzem questões reais.

1. Identificação da aula e recorte do edital

A aula solicitada é a Aula 2 da trilha de Arquitetura de Sistemas e Integração, dedicada aos padrões arquiteturais MVC - Model-View-Controller e DDD - Domain-Driven Design. Ela cobre diretamente o item 1.3 Padrões arquiteturais MVC e DDD do edital de Analista de Sistemas do TJSC 2026. O conteúdo da Aula 1 - arquitetura de software, cliente-servidor, camadas, web e mobile - será usado apenas como base conceitual, sem substituir o foco desta aula.

O objetivo de prova é reconhecer papéis, fronteiras e responsabilidades. A FGV tende a construir alternativas com termos corretos em contexto errado: Controller tratado como Service, Model tratado apenas como tabela, DDD confundido com UML, Repository confundido com DAO mecânico, ou Aggregate usado como nome sofisticado para qualquer classe de domínio.

2. Como a FGV tende a explorar MVC e DDD

Em TI, a FGV raramente se limita ao verbete seco. O padrão provável é um pequeno cenário: um tribunal, órgão público, sistema corporativo, módulo web, API, domínio negocial ou integração entre sistemas. A questão pede o conceito mais adequado ou a incorreta. O candidato experiente erra quando transforma prática de framework em definição de arquitetura.

Para MVC e DDD, o risco central é confundir nível de análise. MVC organiza interação entre modelo, visualização e controle, principalmente na camada de apresentação/web. DDD orienta o desenho do software pelo domínio do negócio, com linguagem ubíqua, contextos delimitados, entidades, objetos de valor, agregados, repositórios e serviços de domínio. Um não substitui o outro: uma aplicação pode usar MVC na borda HTTP e DDD no núcleo de negócio.

Padrão de cobrança	Como pode aparecer	Erro típico
Direta conceitual	Assinalar a função do Controller, do Model ou da View	Marcar Service como Controller ou Model como banco
Caso prático	API Java recebe request e delega para domínio	Achar que Resource/Controller deve conter regra negocial
Afirmativas	Comparar Entity, Value Object, Aggregate, Repository e Service	Tratar todos como camadas técnicas
Exceção/incorreta	Uma alternativa exagera um conceito verdadeiro	Cair em 'sempre', 'apenas', 'automaticamente'
Diagrama textual	Tela -> Controller -> Service -> Repository -> Banco	Dizer que isso por si só é DDD ou microsserviço

3. Siglas e termos essenciais

Sigla/termo	Expansão	Sentido para a prova
MVC	Model-View-Controller	Padrão que separa modelo, visualização e controle, muito comum em aplicações web
DDD	Domain-Driven Design	Abordagem de desenho orientada ao domínio do negócio e à linguagem compartilhada
API	Application Programming Interface	Contrato de interação entre sistemas; na aula, frequentemente exposta por HTTP
HTTP	Hypertext Transfer Protocol	Protocolo de aplicação usado em sistemas web; base de muitos fluxos MVC/REST
UI	User Interface	Interface com o usuário; pode estar no navegador, app mobile ou página renderizada no servidor
DTO	Data Transfer Object	Objeto de transporte de dados entre camadas/processos; não deve virar automaticamente entidade de domínio
ORM	Object-Relational Mapping	Mapeamento objeto-relacional; pode persistir entidades, mas não define por si só DDD
JPA	Java Persistence API	Especificação Java de persistência; será aprofundada na Aula 7
CRUD	Create, Read, Update, Delete	Operações básicas; CRUD organizado não é automaticamente domínio rico
UML	Unified Modeling Language	Linguagem de modelagem; pode apoiar DDD, mas não se confunde com ele

4. MVC - Model-View-Controller

MVC - Model-View-Controller - é um **padrão que separa responsabilidades em três papéis**. A ideia não é dividir o sistema em três servidores, nem impor três camadas físicas. O objetivo é evitar que entrada do usuário, regra/dado e apresentação fiquem misturados no mesmo bloco de código.

Em aplicações web tradicionais, MVC costuma aparecer com um controlador recebendo a requisição, acionando lógica/modelo e escolhendo a visão a ser renderizada. Em frameworks modernos, inclusive Spring MVC, há frequentemente um Front Controller central que recebe a requisição e delega a controladores ou handlers específicos. Em APIs, o controlador pode devolver uma representação JSON em vez de escolher uma página HTML; isso muda a forma da saída, mas não autoriza concentrar regra negocial pesada no Controller.

Componente	Função	O que não é
Model	Representa dados, estado e comportamento relevante para a aplicação ou domínio. Pode envolver entidades, objetos de domínio, DTOs ou estruturas usadas para compor a resposta, conforme arquitetura.	Não é sinônimo obrigatório de tabela de banco, entidade JPA ou DTO.
View	Cuida da apresentação ou representação da saída. Em MVC clássico, pode ser template/página HTML. Em aplicações SPA, a UI pode estar no Angular; em API, a saída é normalmente uma representação, como JSON.	Não executa regra negocial central nem decide fluxo de aplicação.
Controller	Recebe entrada/requisição, interpreta parâmetros, aciona componentes adequados e seleciona resposta ou representação.	Não deve virar depósito de regras de negócio, SQL manual, autorização complexa e orquestração pesada.

4.1 MVC clássico, MVC em APIs e SPA

A FGV pode tentar prender o candidato em uma visão antiga: View igual a página HTML renderizada no servidor. Essa visão é relevante, mas não esgota os cenários atuais. Em uma aplicação corporativa com Angular e backend Java/Quarkus, a View efetiva da experiência do usuário está no front-end, enquanto o backend expõe endpoints que retornam JSON. Mesmo assim, o backend pode ter componentes equivalentes a Controller ou Resource para receber a chamada e coordenar a resposta.

A formulação segura para prova é: REST ou JSON não são MVC. Uma API RESTful pode ser implementada usando componentes com papel de Controller/Resource, Service e Repository, mas REST é estilo de comunicação por recursos, enquanto MVC é padrão de separação entre modelo, visualização e controle. JSON é formato de representação, não padrão arquitetural de apresentação.

Cenário	Como pensar em prova	Armadilha
Spring MVC server-side	DispatcherServlet/Front Controller recebe request, delega a controllers, resolve view e retorna resposta	Achar que DispatcherServlet contém regra de negócio
API REST em Spring ou Quarkus	Controller/Resource recebe HTTP e devolve JSON/erro/status; regra fica em services/domínio	Dizer que JSON é a View clássica sem nuance
Angular + API Java	Angular cuida da UI; API expõe endpoints; ainda há separação de responsabilidades	Achar que usar Angular elimina qualquer ideia de MVC
Monolito em camadas	Pode ter MVC na borda e camadas internas	Chamar MVC de microsserviços

4.2 Exemplo prático: consulta processual

Fluxo textual:

1. Usuário acessa tela Angular e informa número do processo.
2. Angular envia GET /api/processos/{numero}.
3. ProcessoResource ou ProcessoController recebe a requisição.
4. Controller valida formato básico e chama ConsultarProcessoService.
5. Service aplica regra de acesso, consulta domínio/repositório e monta resposta.
6. Repository acessa persistência.
7. API devolve JSON com dados permitidos.
8. Angular renderiza a tela.

Leitura de prova:

- Angular é cliente/UI.
- Controller/Resource não deve conter toda a regra de negócio.
- Service/Application Service orquestra caso de uso.
- Repository abstrai acesso a dados.
- JSON é representação da resposta, não sinônimo de MVC.

4.3 Controller fino, Controller gordo e risco de prova

Controller fino não significa Controller inútil. Ele pode validar aspectos de entrada, lidar com autenticação/autorização técnica conforme o framework, transformar request em comando ou DTO e escolher status HTTP. O problema surge quando o Controller acumula cálculo de prazo, regra de alçada, transação, SQL manual, integração externa e decisão de domínio. Em prova, a alternativa que manda colocar toda regra no Controller para simplificar costuma ser errada.

No seu mundo Java/Quarkus, o nome pode ser Resource em vez de Controller. A função arquitetural é parecida: receber a chamada HTTP e delegar. A FGV pode trocar nome de framework por papel arquitetural; foque no papel, não no nome exato da annotation.

5. DDD - Domain-Driven Design

DDD - Domain-Driven Design - é uma abordagem de desenho de software centrada no domínio do negócio. A ideia central é construir um modelo que reflita conceitos, regras e linguagem do domínio, com colaboração entre especialistas de negócio e desenvolvedores. O DDD não é um framework, não é UML, não é Spring, não é JPA e não é simplesmente criar pacotes chamados domain, service e repository.

Em prova, o ponto mais importante é **distinguir DDD estratégico de DDD tático**. O **estratégico trata de domínio, subdomínios, contexto delimitado e linguagem ubíqua**. O **tático trata de entidades, objetos de valor, agregados, repositórios, serviços de domínio e eventos de domínio**. Ambos se conectam: os padrões táticos só fazem sentido dentro de uma fronteira de modelo bem definida.

Eixo	Foco	Conceitos
DDD estratégico	Fronteiras, linguagem e organização do domínio	Domínio, subdomínio, core domain, bounded context, ubiquitous language, context map
DDD tático	Construção do modelo dentro de um contexto	Entity, Value Object, Aggregate, Aggregate Root, Repository, Domain Service, Domain Event

5.1 Domínio, subdomínio e linguagem ubíqua

Domínio é a área de conhecimento ou atividade que o software precisa atender. Em um tribunal, exemplos de domínio são tramitação processual, distribuição, movimentação, intimação, gestão documental, custas, sessão de julgamento e administração de usuários. Um subdomínio é uma parte desse domínio maior; nem todo subdomínio tem o mesmo valor estratégico.

Linguagem ubíqua é o vocabulário compartilhado entre equipe técnica e especialistas de negócio. Ela evita traduções perigosas: se o negócio fala em 'processo distribuído', 'parte', 'movimentação', 'classe processual' e 'unidade julgadora', o modelo não deveria inventar nomes genéricos como Registro, Item, Objeto e Flag sem necessidade. A FGV pode cobrar isso como alinhamento entre modelo e negócio, não como nomenclatura bonita.

5.2 Bounded Context - contexto delimitado

Bounded Context, ou contexto delimitado, é uma fronteira explícita dentro da qual um modelo e uma linguagem têm significado consistente. O mesmo termo pode ter sentidos diferentes em contextos diferentes. 'Processo' pode significar processo judicial no contexto processual; processo administrativo em gestão interna; processo de negócio em governança. DDD não tenta forçar um modelo único para tudo; ele delimita contextos.

Essa distinção é decisiva para microserviços, mas a Aula 2 não vira aula de microserviços. Para a FGV, guarde: bounded context ajuda a definir fronteiras conceituais; não é automaticamente serviço, banco de dados, pacote Java ou rota de API. Um microserviço pode corresponder a um bounded context, mas essa associação não é obrigatória nem mecânica.

5.3 Entidade

Entity, ou **entidade de domínio**, é um objeto cuja identidade permanece relevante ao longo do tempo, mesmo que seus atributos mudem. Um Processo continua sendo o mesmo processo se muda de unidade, recebe movimentações ou altera situação. Uma Conta continua sendo a mesma conta apesar de saldo variar.

A entidade em DDD não é automaticamente a mesma coisa que entidade JPA. A entidade JPA é uma classe persistida. A entidade de domínio é uma abstração de negócio com identidade e comportamento. Em arquiteturas simples, elas podem coincidir; em arquiteturas mais cuidadosas, podem ser separadas. A prova pode tentar igualar DDD a ORM; essa igualdade é falsa.

5.4 Value Object - objeto de valor

Value Object é definido por seus valores, não por identidade própria. Dois objetos de valor com os mesmos atributos são equivalentes para o domínio. Exemplos: CPF, Endereço, Dinheiro, Período, Email, NúmeroProcessual, Coordenada. Normalmente são imutáveis ou tratados como imutáveis: em vez de alterar um endereço internamente, substitui-se por outro valor.

Critério	Entidade	Value Object
Identidade	Tem identidade própria e continuidade no tempo	Não tem identidade própria; é definido pelos atributos
Mutabilidade	Pode mudar estado mantendo identidade	Preferencialmente imutável
Comparação	Por identidade	Por igualdade de valores
Exemplo TJSC	Processo, Documento, Unidade	Número processual, CPF, Endereço, Período

5.5 Agregado e Aggregate Root

Aggregate, ou agregado, é um **conjunto de objetos de domínio tratado como unidade de consistência**. Ele possui uma raiz, chamada Aggregate Root, por onde as mudanças externas devem ocorrer. A raiz protege invariantes: regras que precisam permanecer verdadeiras após cada operação.

Exemplo: em um agregado Processo, a raiz Processo pode controlar a inclusão de Movimentação, impedindo movimentação em processo arquivado sem reabertura formal. O ponto de prova é: agregado não é qualquer grupo de classes. Ele existe para delimitar consistência, transações e acesso. Objetos internos não devem ser modificados livremente por código externo, pois isso viola invariantes.

5.6 Repository

Repository, no DDD, **representa uma coleção conceitual de agregados e abstrai persistência**. Ele permite obter, salvar ou remover agregados sem expor detalhes de banco, SQL, ORM ou API externa ao domínio. Isso não significa que todo método de Repository deve ser um CRUD genérico, nem que Repository seja sinônimo exato de DAO.

Em Java, um repository pode ser implementado com JPA, Hibernate, Panache, Spring Data, SQL manual ou chamada externa. A abstração pertence ao desenho; a tecnologia de persistência é detalhe de infraestrutura. Em prova, cuidado com alternativa que define Repository como classe que obrigatoriamente contém SQL ou como camada que deve encapsular regras de negócio centrais.

5.7 Service de domínio, Application Service e Infrastructure Service

Um **Service de domínio representa uma operação de negócio que não pertence naturalmente a uma entidade ou objeto de valor específico**. Por exemplo, uma política de distribuição que considera unidade, classe processual, impedimentos e carga de trabalho pode ser expressa como serviço de domínio se a regra não couber naturalmente em uma única entidade.

Application Service, por sua vez, orquestra um caso de uso: abre transação, carrega agregados, chama métodos do domínio, aciona repositórios e prepara resposta. Ele não deve concentrar regra negocial central se essa regra pertence ao domínio. Infrastructure Service encapsula integração técnica, como envio de email, acesso a fila, client REST, armazenamento externo ou autenticação.

Tipo	Responsabilidade	Erro comum
Domain Service	Regra de negócio sem dono natural em uma entidade/value object	Virar classe utilitária genérica ou anêmica
Application Service	Orquestrar caso de uso, transação e fluxo da aplicação	Concentrar toda regra de negócio e deixar entidades sem comportamento
Infrastructure Service	Detalhes técnicos: email, broker, REST client, armazenamento, segurança técnica	Vazar detalhes técnicos para o domínio

5.8 Domain Event

Domain Event, ou evento de domínio, representa **algo relevante que aconteceu no domínio**: ProcessoDistribuido, DocumentoAssinado, PagamentoAprovado, AudienciaMarcada. Ele não é apenas mensagem de RabbitMQ; é primeiro um fato do domínio. Pode ou não ser publicado em mensageria. Esse assunto será aprofundado na Aula 8, mas aqui já importa separar evento de domínio de evento técnico.

6. MVC e DDD juntos em uma arquitetura corporativa

MVC e DDD atuam em níveis diferentes e podem coexistir. Em uma API Java, Controller/Resource fica na borda de entrada HTTP; Application Service orquestra o caso de uso; Domain Model contém entidades, value objects, agregados e serviços de domínio; Repository abstrai persistência; infraestrutura implementa detalhes técnicos. O erro típico é chamar tudo de MVC ou chamar todo pacote domain de DDD.

Elemento no código	Leitura arquitetural	Risco em questão
ProcessoController / ProcessoResource	Borda HTTP; recebe request e retorna response	Colocar regra negocial pesada nele
ConsultarProcessoUseCase	Application Service; coordena caso de uso	Confundir com Domain Service automaticamente
Processo	Entidade/agregado se tiver identidade e regras	Tratar como DTO anêmico sem comportamento
NumeroProcessual	Value Object se definido por valor e regras de validação	Modelar como string solta em todo lugar
ProcessoRepository	Abstração de persistência de agregados	Confundir com DAO obrigatoriamente SQL
JpaProcessoRepository	Implementação de infraestrutura	Vazar JPA/Hibernate para o domínio

6.1 Exemplo textual: distribuição de processo

Cenário:

Um sistema precisa distribuir um processo para uma unidade competente.

A regra considera classe processual, competência, impedimentos, carga e prevenção.

Possível desenho:

- DistribuirProcessoController recebe POST /processos/{id}/distribuicao.
- DistribuirProcessoUseCase carrega Processo, Unidade e regras necessárias.
- PoliticaDeDistribuicao é Domain Service se a regra não pertence a uma única entidade.
- Processo é Aggregate Root se controla estado e invariantes da distribuição.
- NumeroProcessual pode ser Value Object.
- ProcessoRepository carrega e salva o agregado.
- JpaProcessoRepository ou PanacheProcessoRepository implementa persistência.

Ponto de prova:

O fluxo usa Controller, mas a regra de distribuição não deve ficar no Controller.

O uso de Repository não prova DDD se o domínio for apenas DTO + CRUD.

DDD exige modelo com linguagem, fronteira e comportamento do domínio.

7. Pegadinhas FGV sobre MVC e DDD

Pegadinha	Correção para prova
MVC é uma arquitetura de três camadas	MVC tem três papéis; camadas são outra forma de organização lógica. Podem coexistir, mas não são sinônimos.
Model é sempre tabela do banco	Model representa estado/dados/comportamento da aplicação; pode ou não corresponder a tabela.
Controller deve conter regras de negócio	Controller deve coordenar entrada/saída e delegar regra central para serviços/domínio.
API REST com JSON deixa de usar MVC	REST/JSON são comunicação/representação; podem existir controllers/resources na borda.
DDD é usar pasta domain	DDD exige modelo orientado ao domínio, linguagem ubíqua, contextos e padrões táticos bem aplicados.
Entity DDD é a mesma coisa que entidade JPA	Podem coincidir, mas pertencem a preocupações diferentes: domínio versus persistência.
Value Object é objeto sem importância	Value Object carrega regras e significado, mas é definido por valor, não por identidade.
Aggregate é qualquer composição de objetos	Agregado delimita consistência e protege invariantes por meio da raiz.
Repository é DAO com outro nome	Pode haver semelhança, mas Repository no DDD abstrai coleção de agregados e isola persistência.
Domain Service é todo Service	Service de domínio contém regra de negócio sem dono natural; Application Service orquestra caso de uso.

8. Checklist conceitual para resolver questão

Ao ler uma questão da FGV, classifique mentalmente o problema antes de olhar as alternativas:

- Se a pergunta é sobre entrada HTTP, seleção de resposta ou encaminhamento para handler, pense em Controller, Front Controller e MVC.
- Se a pergunta é sobre apresentação ao usuário ou representação de saída, pense em View/representação, mas não transforme JSON em View clássica sem nuance.
- Se a pergunta é sobre identidade, ciclo de vida e comportamento do negócio, pense em Entity.
- Se a pergunta é sobre igualdade por atributos e imutabilidade, pense em Value Object.
- Se a pergunta é sobre consistência transacional e invariantes internas, pense em Aggregate/Aggregate Root.
- Se a pergunta é sobre coleção conceitual de agregados e isolamento de persistência, pense em Repository.
- Se a pergunta é sobre operação de negócio sem dono natural, pense em Domain Service; se é orquestração de caso de uso, pense em Application Service.
- Se a alternativa usa sempre, apenas, automaticamente ou obrigatoriamente, teste se há exceção técnica razoável.

9. Lista de questões autorais - padrão FGV

Resolva as questões sem consultar o gabarito. As alternativas foram construídas com distratores tecnicamente próximos. O gabarito e os comentários completos aparecem depois da lista.

1. Em um sistema web do TJSC, uma requisição HTTP chega a um componente central do framework, que aplica uma rotina comum de processamento e delega a chamada para o controlador apropriado. Considerando o padrão MVC em frameworks web, esse componente central se aproxima do padrão:

- (A) Repository, pois centraliza a persistência de entidades de domínio.
- (B) Value Object, pois encapsula a requisição HTTP como valor imutável.
- (C) Service Layer, pois substitui os controllers específicos por regras de negócio reutilizáveis.
- (D) Front Controller, pois concentra a entrada inicial da requisição e delega a componentes adequados.
- (E) Aggregate Root, pois garante a consistência transacional entre objetos do domínio.

2. No padrão MVC - Model-View-Controller -, assinale a alternativa que melhor descreve a responsabilidade típica do Controller em uma aplicação web.

- (A) Persistir entidades diretamente no banco e definir índices físicos.
- (B) Renderizar exclusivamente componentes visuais no navegador, sem interação com o servidor.
- (C) Representar a identidade de objetos do domínio, garantindo invariantes de agregados.
- (D) Receber a entrada ou requisição, coordenar o fluxo e acionar componentes necessários à resposta.
- (E) Substituir a camada de domínio, concentrando todas as regras negociais da aplicação.

3. Uma equipe afirma que, por utilizar classes Controller, Service e Repository em um monolito Java, a aplicação necessariamente adota DDD. A afirmação é:

- (A) correta, pois DDD é definido pela presença dessas três classes em qualquer framework Java.
- (B) incorreta, pois nomes de camadas ou pacotes não bastam; DDD exige modelo orientado ao domínio, linguagem e fronteiras consistentes.
- (C) correta, desde que o Controller exponha endpoints HTTP em formato JSON.
- (D) incorreta apenas porque DDD só pode ser usado em microsserviços.
- (E) correta, pois Repository é sinônimo obrigatório de DDD e torna dispensável a modelagem do domínio.

4. Considere as afirmativas sobre DDD. I. Uma entidade de domínio é relevante por sua identidade e continuidade ao longo do tempo. II. Um objeto de valor é definido por seus atributos e, em geral, pode ser tratado como imutável. III. Um agregado define uma fronteira de consistência e deve possuir uma raiz que controle acesso externo. Está correto o que se afirma em:

- (A) I, apenas.
- (B) II, apenas.
- (C) I e II, apenas.
- (D) II e III, apenas.
- (E) I, II e III.

5. Em um sistema de processos, NumeroProcessual possui regras de validação próprias, é comparado por valor e não possui ciclo de vida independente de um Processo. Em DDD, esse conceito tende a ser modelado como:

- (A) Value Object.
- (B) Entity Root, pois todo número possui identidade própria no banco.
- (C) Repository, pois encapsula consultas ao banco.

(D) Controller, pois recebe dados externos da requisição.

(E) Bounded Context, pois representa uma fronteira organizacional completa.

6. Em uma API REST, o método de um Controller contém validações de negócio complexas, cálculo de prazos processuais, consulta SQL manual e decisão de alçada. À luz de MVC e separação de responsabilidades, a crítica mais adequada é que:

(A) o Controller deveria ser substituído por uma View, pois APIs não admitem controle de fluxo.

(B) a API deixou de ser cliente-servidor por possuir regras de negócio.

(C) o Controller acumula responsabilidades que deveriam ser distribuídas entre aplicação/domínio e persistência.

(D) a existência de SQL no Controller prova que a arquitetura é DDD tático completo.

(E) o problema é exclusivamente o uso de JSON, que é incompatível com MVC.

7. Sobre Bounded Context em DDD, assinale a opção correta.

(A) É uma tabela relacional que armazena todos os conceitos do domínio para evitar duplicidade semântica.

(B) É um pacote Java obrigatório chamado bounded.context, exigido por frameworks DDD.

(C) É sempre equivalente a um microserviço implantado em contêiner próprio.

(D) É uma fronteira explícita dentro da qual um modelo e uma linguagem têm significado consistente.

(E) É a camada de apresentação responsável por renderizar views em MVC clássico.

8. Relacione os conceitos de DDD às descrições. 1. Entity 2. Value Object 3. Repository 4. Domain Service () Operação de negócio relevante que não pertence naturalmente a uma entidade ou objeto de valor. () Objeto distinguido por identidade e continuidade. () Abstração de coleção/persistência de agregados. () Objeto definido por seus valores. A sequência correta é:

(A) 1 - 2 - 3 - 4.

(B) 4 - 1 - 3 - 2.

(C) 3 - 1 - 4 - 2.

(D) 4 - 3 - 1 - 2.

(E) 2 - 1 - 3 - 4.

9. Uma aplicação Angular consome uma API Java/Quarkus que retorna JSON. A equipe conclui que, por não haver páginas HTML renderizadas no servidor, nenhum elemento do padrão MVC pode ser reconhecido na arquitetura. A conclusão é:

(A) incorreta, pois ainda pode haver componente com papel de Controller/Resource recebendo requisições e coordenando respostas.

(B) correta, pois MVC exige obrigatoriamente JSP ou Thymeleaf.

(C) correta, pois JSON é incompatível com separação entre entrada, modelo e resposta.

(D) incorreta apenas se a API estiver implantada em Kubernetes.

(E) correta, pois Angular elimina a necessidade de qualquer separação de responsabilidades no backend.

10. Assinale a alternativa INCORRETA sobre MVC.

(A) O Controller deve, como regra, concentrar toda a regra de negócio para reduzir o número de classes do sistema.

(B) O Model não deve ser entendido automaticamente como tabela física do banco de dados.

(C) A View está associada à apresentação ou representação da saída.

(D) O padrão MVC pode coexistir com arquitetura em camadas.

(E) Em frameworks web, pode haver um Front Controller que centraliza a entrada inicial de requisições.

11. Em DDD, um Aggregate Root deve:

- (A) ser sempre uma entidade JPA anotada com @Entity e acessada diretamente por qualquer controller.
- (B) permitir que objetos internos sejam alterados livremente para reduzir acoplamento.
- (C) controlar o acesso externo ao agregado e proteger invariantes de consistência.
- (D) substituir o Repository, evitando qualquer abstração de persistência.
- (E) representar exclusivamente uma View de domínio em MVC.

12. Observe o diagrama textual: Tela Angular -> ProcessoController -> DistribuirProcessoUseCase -> PoliticaDeDistribuicao -> ProcessoRepository. A interpretação mais adequada é:

- (A) ProcessoController é o agregado raiz, pois aparece logo após a tela.
- (B) DistribuirProcessoUseCase tende a atuar como Application Service, orquestrando o caso de uso.
- (C) PoliticaDeDistribuicao é obrigatoriamente Repository por conter regra de consulta.
- (D) ProcessoRepository deve conter as regras centrais de distribuição.
- (E) A presença de Angular torna impossível aplicar princípios de DDD no backend.

13. Assinale a alternativa que melhor diferencia Domain Service de Application Service em DDD.

- (A) Domain Service trata de endpoints HTTP; Application Service trata de templates HTML.
- (B) Domain Service é qualquer classe anotada com @Service; Application Service é sempre uma fila.
- (C) Domain Service implementa persistência; Application Service representa value objects.
- (D) Domain Service expressa regra de negócio sem dono natural em entidade/value object; Application Service orquestra caso de uso.
- (E) Domain Service substitui o domínio; Application Service substitui o Controller.

14. Julgue as afirmativas sobre Repository em DDD. I. Repository deve esconder detalhes de persistência do domínio. II. Repository é obrigatoriamente uma classe com SQL manual. III. Repository pode ser implementado por tecnologia de infraestrutura, como ORM, sem que essa tecnologia defina o domínio. Assinale a opção correta.

- (A) Apenas I está correta.
- (B) Apenas II está correta.
- (C) Apenas III está correta.
- (D) I e II estão corretas.
- (E) I e III estão corretas.

15. Uma equipe modela Pedido como entidade sem comportamento, com apenas getters e setters, e coloca todas as regras de validação, cálculo e mudança de estado em classes externas procedurais. Considerando DDD, isso se aproxima do problema conhecido como:

- (A) Front Controller.
- (B) Bounded Context.
- (C) Anemic Domain Model.
- (D) View Resolver.
- (E) Value Object imutável.

16. Sobre a relação entre DDD e microsserviços, assinale a alternativa mais adequada.

- (A) DDD só existe quando cada entidade vira um microsserviço independente.
- (B) DDD é incompatível com monólitos, pois exige deployment independente por agregado.

- (C) DDD pode ajudar a identificar fronteiras, mas bounded context não é automaticamente sinônimo de microserviço.
- (D) Microserviços dispensam linguagem ubíqua, pois a comunicação ocorre por API.
- (E) Todo Repository deve ser exposto como API pública para caracterizar DDD.

17. No contexto de MVC, a View é melhor compreendida como:

- (A) o componente que aplica invariantes do agregado.
- (B) a apresentação ou representação da saída ao usuário ou cliente.
- (C) a abstração de persistência de entidades.
- (D) a fronteira estratégica de linguagem do domínio.
- (E) o componente que decide, sozinho, regras de transação e autorização negocial.

18. Um especialista do negócio usa o termo 'movimentação' para atos processuais registrados em determinada fase, enquanto outra área usa 'movimentação' para deslocamento físico de bens. Em DDD, a forma mais adequada de lidar com essa diferença é:

- (A) forçar uma tabela única MOVIMENTACAO para todos os sentidos do termo.
- (B) renomear tudo para Evento, pois termos genéricos evitam conflito.
- (C) eliminar o termo do modelo, pois ambiguidade impede modelagem.
- (D) delimitar contextos e manter linguagem consistente dentro de cada fronteira.
- (E) resolver exclusivamente com herança de classes no Controller.

19. Em um sistema de custas, Dinheiro é composto por valor e moeda, deve ser comparado por seus atributos e não possui identidade independente. Pela ótica de DDD, o conceito é mais aderente a:

- (A) Domain Service, pois representa cálculo de valor.
- (B) Entity, pois todo objeto com atributos deve possuir identidade.
- (C) Controller, pois vem de uma requisição externa.
- (D) Repository, pois precisa ser armazenado.
- (E) Value Object, pois é definido por valor e pode ser tratado como imutável.

20. Uma questão apresenta a seguinte descrição: 'O modelo deve refletir a linguagem dos especialistas do domínio; conceitos centrais devem carregar comportamento; a persistência deve ser isolada por abstrações; regras que protegem consistência devem estar nos agregados'. Essa descrição se associa principalmente a:

- (A) MVC estritamente server-side.
- (B) Modelo relacional físico.
- (C) Protocolo HTTP stateless.
- (D) DDD aplicado ao desenho do domínio.
- (E) View resolver de templates.

10. Gabarito resumido

Questão	Gabarito	Questão	Gabarito	Questão	Gabarito	Questão	Gabarito
1	D	6	C	11	C	16	C
2	D	7	D	12	B	17	B
3	B	8	B	13	D	18	D
4	E	9	A	14	E	19	E
5	A	10	A	15	C	20	D

Distribuição orgânica: A = 4, B = 4, C = 4, D = 5, E = 3. O gabarito não foi calibrado por quota matemática; as alternativas foram embaralhadas por plausibilidade e revisadas para evitar padrão visual óbvio.

11. Comentários completos

Questão 1 - Gabarito: D

O enunciado descreve um componente central que recebe a entrada inicial da requisição e delega o processamento para handlers/controladores, papel típico do Front Controller em frameworks MVC web. (A) parece tentadora porque Repository também centraliza uma responsabilidade, mas a responsabilidade é persistência, não entrada HTTP. (B) usa termo real de DDD, mas Value Object não coordena requisições. (C) Service Layer pode orquestrar casos de uso, mas não é o componente central de despacho de requests do MVC. (E) Aggregate Root protege invariantes de domínio, não roteia requisições web.

Questão 2 - Gabarito: D

Controller recebe entrada ou requisição, interpreta parâmetros, coordena o fluxo inicial e aciona componentes para produzir resposta. (A) desloca persistência e índices físicos para o Controller, confundindo com infraestrutura. (B) descreve mais a UI/View, ainda de modo limitado. (C) fala de entidade/agregado DDD, não de Controller MVC. (E) é a armadilha clássica: Controller não deve concentrar toda regra negocial.

Questão 3 - Gabarito: B

Apenas usar nomes Controller, Service e Repository não comprova DDD. DDD exige modelo orientado ao domínio, linguagem ubíqua, fronteiras coerentes e padrões táticos aplicados com sentido. (A) reduz DDD a nomenclatura de classes. (C) JSON/HTTP dizem respeito à API, não à qualidade do domínio. (D) erra porque DDD pode existir em monólitos. (E) supervaloriza Repository: ele ajuda, mas não torna dispensável a modelagem do domínio.

Questão 4 - Gabarito: E

As três afirmativas estão corretas. Entidade é relevante por identidade; Value Object é definido por valores; agregado delimita consistência e possui raiz. (A), (B), (C) e (D) selecionam subconjuntos verdadeiros, mas incompletos. O cuidado FGV é perceber que os três conceitos são compatíveis e complementares, não concorrentes.

Questão 5 - Gabarito: A

NumeroProcessual é comparado por valor, tem regra própria e não tem ciclo de vida independente; isso é típico de Value Object. (B) confunde todo dado importante com entidade. (C) Repository é abstração de persistência. (D) Controller é borda de entrada. (E) Bounded Context é fronteira de modelo e linguagem, não um valor específico.

Questão 6 - Gabarito: C

O Controller está gordo: mistura entrada HTTP, regra negocial complexa e persistência. Boas práticas de MVC e camadas recomendam delegar regra para aplicação/domínio e persistência para repositórios/infraestrutura. (A) View não substitui Controller. (B) cliente-servidor continua existindo. (D) SQL no Controller não prova DDD; costuma indicar acoplamento. (E) JSON é formato de representação e não causa o problema.

Questão 7 - Gabarito: D

Bounded Context é fronteira explícita de modelo e linguagem. (A) confunde com tabela única; DDD não busca necessariamente unificar tudo. (B) transforma conceito em convenção de pacote. (C) é uma meia verdade perigosa: pode orientar microserviços, mas não é sempre equivalente. (E) confunde DDD estratégico com MVC.

Questão 8 - Gabarito: B

Domain Service é operação de negócio sem dono natural; Entity tem identidade; Repository abstrai coleção/persistência; Value Object é definido por valores. (A) parece plausível se o candidato decorar ordem dos conceitos, mas troca serviço e entidade. (C) coloca Repository como operação de negócio. (D) troca Entity e Repository. (E) coloca Value Object como operação de negócio, erro comum quando 'valor' parece ação calculada.

Questão 9 - Gabarito: A

Mesmo sem HTML server-side, o backend pode ter componente Controller/Resource que recebe HTTP e coordena a resposta JSON. (B) limita MVC a tecnologias específicas. (C) confunde JSON com impossibilidade de separação de responsabilidades. (D) Kubernetes é irrelevante para o papel MVC. (E) Angular não elimina a necessidade de organização no backend.

Questão 10 - Gabarito: A

A alternativa incorreta é A. Controller não deve, como regra, concentrar toda regra de negócio. (B) está correta: Model não é automaticamente tabela. (C) está correta: View cuida de apresentação/representação. (D) está correta: MVC pode coexistir com camadas. (E) está correta: frameworks podem usar Front Controller.

Questão 11 - Gabarito: C

Aggregate Root controla o acesso externo ao agregado e protege invariantes. (A) confunde agregado com entidade JPA obrigatória e ainda permite acesso direto pelo Controller. (B) viola encapsulamento e invariantes. (D) Repository continua necessário como abstração de persistência. (E) mistura DDD com View de MVC.

Questão 12 - Gabarito: B

DistribuirProcessoUseCase coordena um caso de uso, papel típico de Application Service. (A) Controller não vira agregado raiz por posição no fluxo. (C) PoliticaDeDistribuicao parece Domain Service, não Repository, se contém regra de distribuição. (D) Repository não deve conter regra central do domínio. (E) Angular não impede DDD no backend.

Questão 13 - Gabarito: D

Domain Service expressa regra de negócio sem dono natural em entidade/value object; Application Service orquestra o caso de uso. (A) confunde com MVC. (B) reduz conceito a annotation e ainda transforma Application Service em fila. (C) troca Domain Service por persistência e Application Service por Value Object. (E) exagera: nenhum dos dois substitui integralmente domínio ou Controller.

Questão 14 - Gabarito: E

I e III estão corretas. Repository isola persistência do domínio e pode ser implementado por tecnologias de infraestrutura sem defini-las como domínio. II está errada: SQL manual é uma possibilidade de implementação, não obrigação. (A) ignora a correção de III. (B) e (C) selecionam itens isolados inadequadamente. (D) inclui a II errada.

Questão 15 - Gabarito: C

O cenário descreve Anemic Domain Model: objetos com nomes do domínio, mas quase sem comportamento, enquanto regras ficam em serviços procedurais. (A) Front Controller é padrão MVC web. (B) Bounded Context é fronteira estratégica. (D) View Resolver resolve apresentação. (E) Value Object imutável é o oposto de entidade anêmica e não explica regras externas.

Questão 16 - Gabarito: C

DDD pode ajudar a identificar fronteiras, mas bounded context não é automaticamente microserviço. (A) é fragmentação exagerada e errada. (B) DDD pode existir em monólitos modulares. (D) APIs não substituem linguagem ubíqua; contrato técnico não resolve semântica do negócio. (E) Repository não deve ser exposto automaticamente como API pública.

Questão 17 - Gabarito: B

View está associada à apresentação ou representação da saída. (A) é papel de agregado/entidade de domínio. (C) descreve Repository. (D) descreve Bounded Context. (E) atribui à View decisões de transação e autorização negocial, o que viola separação de responsabilidades.

Questão 18 - Gabarito: D

DDD recomenda delimitar contextos e manter linguagem consistente dentro de cada fronteira. (A) força modelo único e gera ambiguidade. (B) genericidade excessiva apaga linguagem do domínio. (C) ambiguidade não impede modelagem; exige delimitação. (E) herança no Controller não resolve diferença semântica de domínio.

Questão 19 - Gabarito: E

Dinheiro é definido por valor e moeda, comparado por atributos e sem identidade própria; é Value Object. (A) Domain Service seria operação de domínio, não o conceito Dinheiro. (B) nem todo objeto com atributos é entidade. (C) vir da requisição não define papel arquitetural. (D) poder ser persistido não transforma algo em Repository.

Questão 20 - Gabarito: D

A descrição reúne linguagem ubíqua, comportamento no domínio, isolamento de persistência e agregados protegendo consistência: núcleo de DDD. (A) MVC server-side trata de apresentação/controle, não todo esse desenho de domínio. (B) modelo físico relacional trata de implementação. (C) HTTP stateless é protocolo. (E) View resolver é componente de apresentação.

12. Checklist final da Aula 2

Ponto	Situação
Item do edital 1.3 - MVC e DDD	Coberto diretamente
MVC	Papéis, variações web/API/SPA, Front Controller, Controller fino e pegadinhas
DDD estratégico	Domínio, subdomínio, linguagem ubíqua e bounded context
DDD tático	Entity, Value Object, Aggregate, Aggregate Root, Repository, Domain Service, Application Service e Domain Event
Exemplos práticos	Angular, Java/Quarkus, API, sistema processual e distribuição de processo
Questões	20 questões autorais com formatos variados e comentários completos
Acessibilidade	Fundo claro, texto preto, sem faixas escuras

13. Referências técnicas utilizadas

- Edital retificado TJSC 2026 - FGV Conhecimento - Analista de Sistemas - Arquitetura de Sistemas e Integração.
- Spring Framework Documentation - Web MVC / DispatcherServlet / Front Controller.
- Microsoft Learn - Domain-Driven Design, entidades, objetos de valor, agregados, repositórios e serviços.
- Microsoft Learn / Azure Architecture - Tactical DDD and microservices boundaries.
- Martin Fowler - Domain-Driven Design, Bounded Context and Anemic Domain Model.
- Eric Evans - Domain-Driven Design: Tackling Complexity in the Heart of Software - referência conceitual clássica para DDD.

Observação: as fontes foram usadas para calibrar conceitos, vocabulário técnico e nível de cobrança. As questões são autorais e não reproduzem itens reais.